
Next.js Full-Stack Development Guide

CONTENTS

Catalog

01

Strategic Introduction

02

Architectural Revolution

03

Performance and Data
Strategy

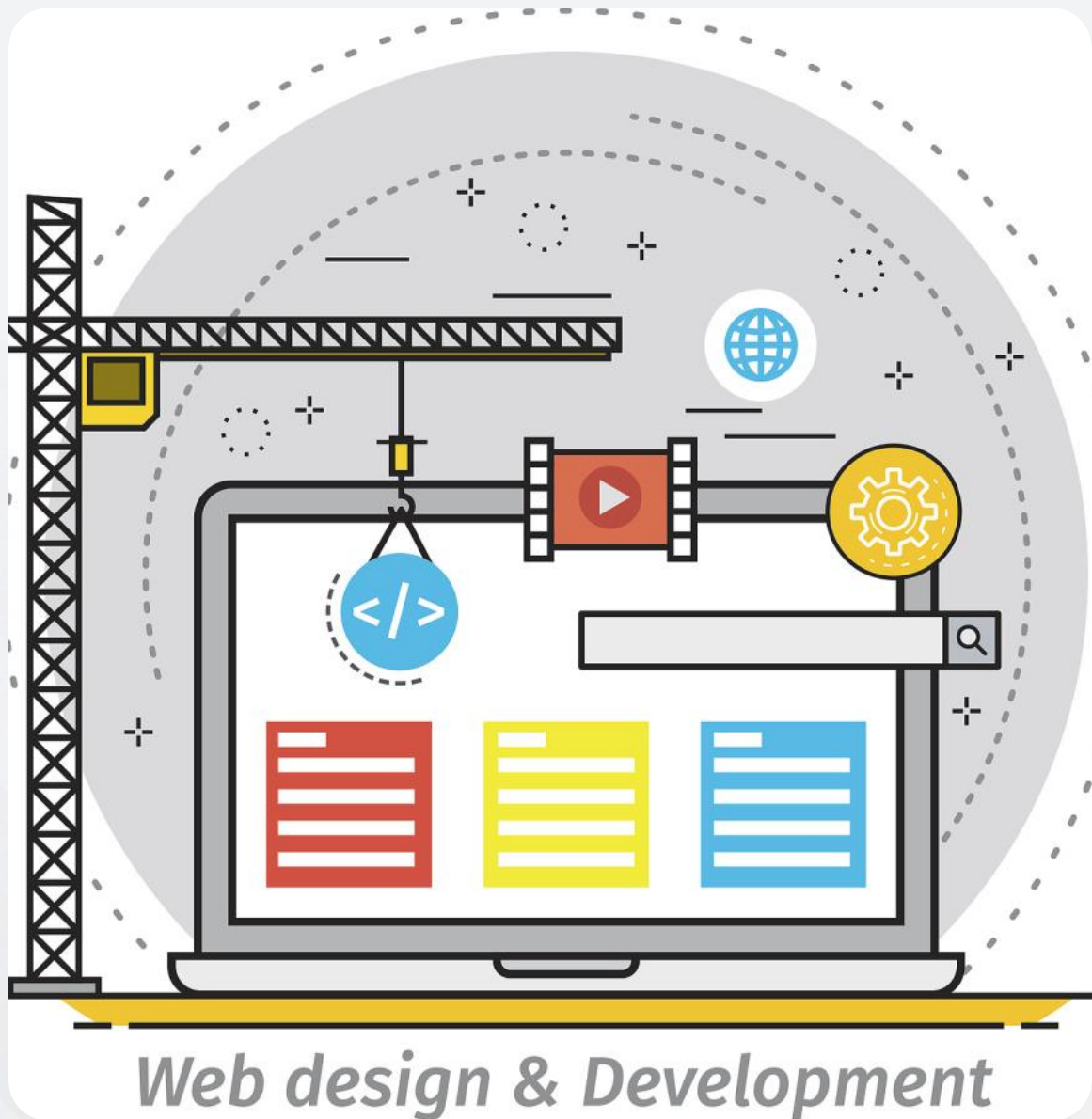
04

Professionalism and
Deployment Readiness



01

Strategic Introduction



Goal: High-performance web apps

Building high-performance apps

The goal is to create web applications that are high-performing, deployable, and scalable.

Deployable and scalable

Ensuring that the web applications are not only high-performing but also easily deployable and scalable.

Why Next.js for SSR



Server-Side Rendering (SSR) for speed

Next.js is chosen for its ability to enhance speed and SEO through Server-Side Rendering.



SEO benefits

Next.js's SSR capabilities also contribute to better search engine optimization.



Next.js 13's App Router shift

Moving from Pages Router to App Router

Next.js 13 introduces a shift from the traditional Pages Router to the new App Router.

Folder-based routing shift

The new App Router is based on a folder structure, which simplifies routing and improves development efficiency.



02

Architectural Revolution

Server Components: Advantages & Use Cases

Zero JavaScript Bundle Size

Server Components run on the server, eliminating the need for client-side JavaScript bundles.

Direct Access to Backend

They can access backend secrets and databases directly, enhancing security and performance.

Faster Initial Load

Server Components contribute to a faster Time To First Byte (TTFB) due to their server-only execution.

Client Components: Use Cases



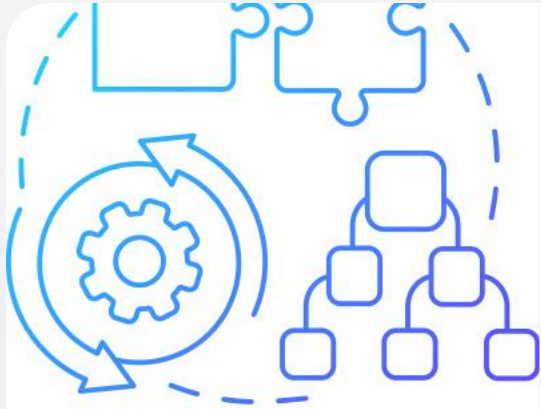
Interactivity Requirements

Client Components are used for interactive elements, such as event handlers and state management.

Browser-specific APIs

They are optimal for leveraging Browser APIs like window or onClick, which are not available server-side.

Composition Strategy: 'Islands' Model



Combining Server & Client Components

The 'Islands' model integrates small Client Components within larger Server Components for optimal performance.



THE CUSTOM-
CONFIGURED MODEL

Improving User Experience

This approach allows for immediate visual feedback and interactive elements, enhancing the overall UX.



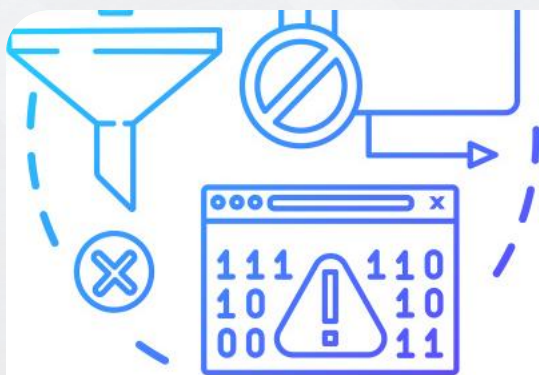
03

Performance and Data Strategy

Data Fetching in Server Components

Direct Data Fetching in Server Components

Next.js allows fetching data directly in Server Components, eliminating the need for `useEffect` boilerplate on the server.



Faster Initial Load

Server Components run on the server only, resulting in a faster initial load time (TTFB).

Caching and Revalidation Strategies

Automatic Caching of Fetch Requests

Next.js automatically caches fetch requests, reducing duplication and improving performance.

Static vs. Dynamic Rendering

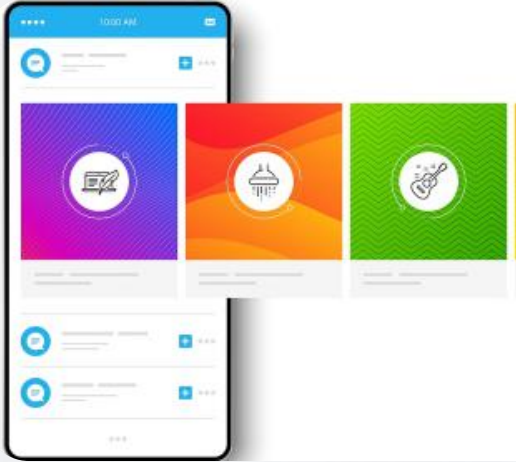
Deciding where and when to cache data is crucial for performance, with Next.js providing options for both static and dynamic rendering.

Revalidation for Fresh Data

Revalidation ensures that data, such as AI model metrics, stays fresh and up-to-date.



Streaming and Loading States for UX



Immediate Visual Feedback

Next.js improves user experience by providing immediate visual feedback through streaming.



Using loading.tsx for Skeletons

loading.tsx is used to display skeletons while waiting for slow data fetches, enhancing the user experience.



04

Professionalism and Deployment Readiness

TypeScript for Type Safety

TypeScript as Standard

TypeScript ensures type safety and scalability, catching errors early.

Benefits of TypeScript

Better refactoring, end-to-end type safety, and improved developer experience.

Server Actions for Data Mutation



Server Actions for Mutations

Server Actions provide a modern solution for form submission and data mutation.



Progressive Enhancement

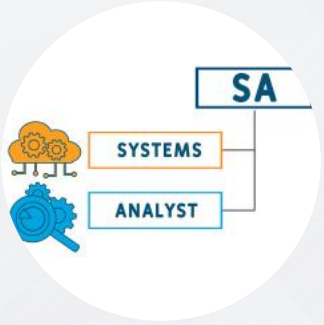
Forms remain functional without JavaScript, enhancing user experience.



Caching Integration

Automatic revalidation occurs after successful mutations, ensuring data freshness.

Authentication & Styling Solutions



Authentication with NextAuth

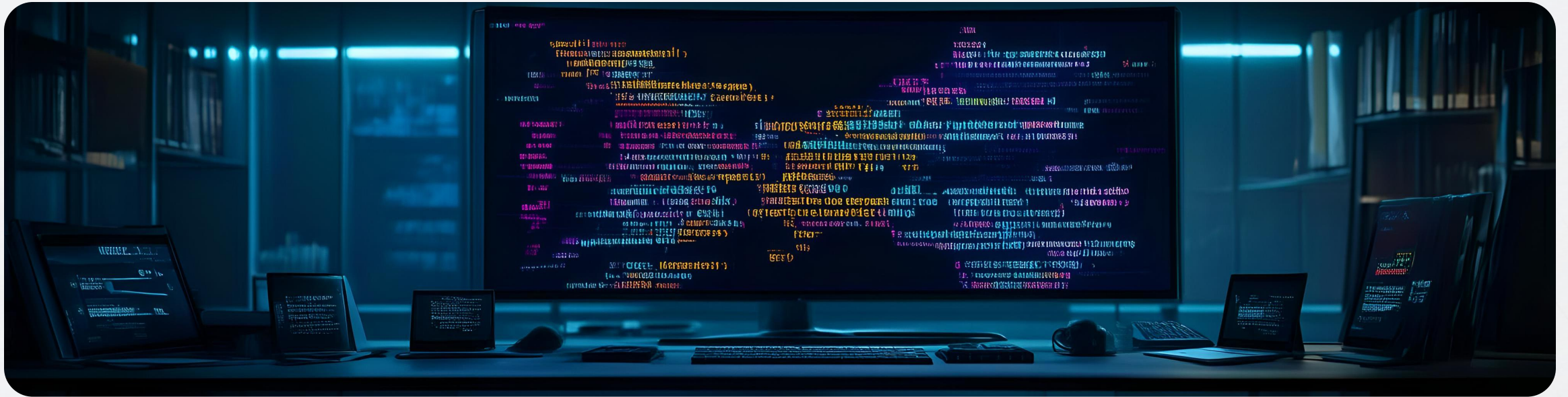
Protecting routes with NextAuth, providing secure authentication solutions.



Modern Styling with Tailwind CSS

Utilizing Tailwind CSS for modern styling, enhancing the visual appeal of applications.

Next.js for MLOps/SaaS Production



Integrated Full-Stack Tool

Next.js meets MLOps/SaaS production requirements as a comprehensive full-stack solution.

MLOps/SaaS Production

Readiness

Next.js is tailored for production environments, supporting machine learning operations and SaaS platforms.

THE END

Thank You